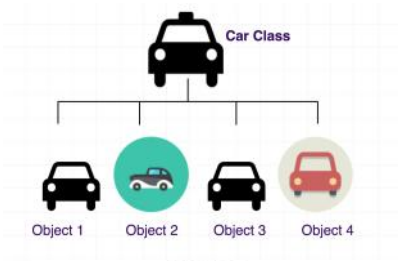
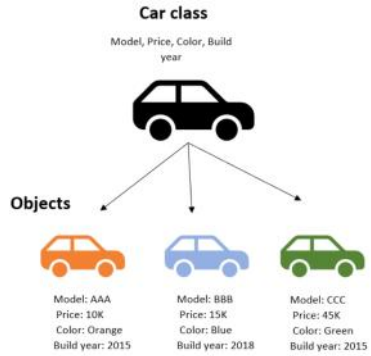
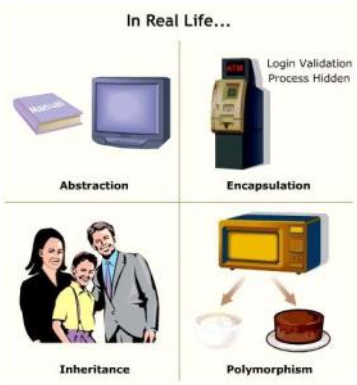
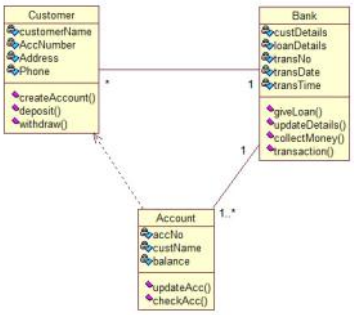
OOPS Complete course

15 April 2026 18:49









Who Should Learn OOP?

1. College Students (CS/IT)

- Core subject
- Important for exams
- Foundation for advanced subjects

2. Placement Preparation

- Asked in almost every interview
- Especially in Java, C++, Python roles

3. Developers

- Backend developers
- Android developers
- Game developers
- System designers

4. When Do We Use OOP?

Use OOP when:

- System is **large and complex**
- You need **modularity**
- You want **clean and maintainable code**

Problem Statement

Imagine you are building a **College Management System**:
You need to manage:

- Students
- Teachers
- Courses
- Attendance
- Results

If you write everything in functions (procedural style):

- Data will be scattered
- Hard to manage
- Difficult to scale

👉 This is where **OOP comes in**

2. Why OOP?

Core Reasons

1. Real-World Mapping

OOP lets you model real-world entities:

- Student → object
- Teacher → object
- Course → object

2. Code Organization

Instead of messy code:

OOP lets you model real-world entities:

- Student → object
- Teacher → object
- Course → object

2. Code Organization

Instead of messy code:

- Group data + behavior together

3. Reusability

Write once, reuse multiple times
(using inheritance)

4. Security

Hide sensitive data
(using encapsulation)

5. Scalability

Easily extend systems
(add new features without breaking old code)

5. Simple Example to Introduce OOP

Without OOP (Procedural Thinking)

```
String studentName = "Shivansh";
int marks = 90;
void printResult() {
    System.out.println(studentName + " scored " + marks);
}
```

✗ Problem:

- Data and behavior are separate
- Not scalable for multiple students

With OOP

```
class Student {
    String name;
    int marks;
void printResult() {
        System.out.println(name + " scored " + marks);
    }
}
Student s1 = new Student();
s1.name = "Shivansh";
s1.marks = 90;
s1.printResult();
```

☑ Benefits:

- Clean structure
- Easy to extend
- Multiple objects possible

1. What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects**, which contain:

- **Data (attributes / properties)**
- **Behavior (methods / functions)**

Instead of writing logic as procedures, OOP organizes code around real-world entities.

Example (real-world mapping):

- Car → Object
- Properties → color, speed
- Methods → drive(), brake()

2. Procedural vs OOP

Feature	Procedural Programming	OOP
Approach	Functions	Objects
Data Handling	Separate from functions	Bundled with methods
Reusability	Low	High
Example	C	Java, C++, Python

3. Class and Object

Class

Blueprint/template for creating objects.

Object

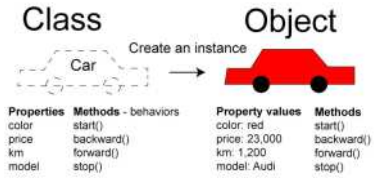
Instance of a class.

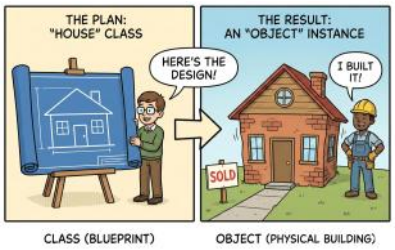


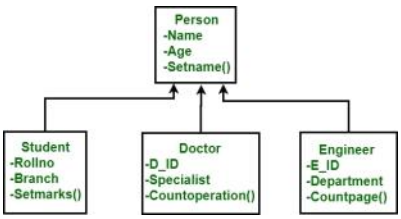
Blueprint/template for creating objects.

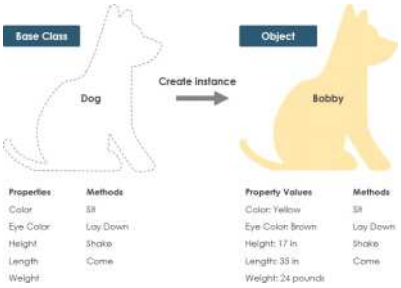
Object

Instance of a class.









Example (Java-like pseudocode)

```
class Car {
    String color;
    int speed;
void drive() {
        System.out.println("Car is moving");
    }
}
Car c1 = new Car();
c1.color = "Red";
c1.speed = 100;
c1.drive();
```

4. Four Pillars of OOP

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

5. Encapsulation

Definition

Wrapping data and methods into a single unit and restricting direct access.

Why?

- Data hiding
- Security
- Controlled access

Example

```
class BankAccount {
    private int balance;
public void deposit(int amount) {
        balance += amount;
    }
public int getBalance() {
        return balance;
    }
}
```

Explanation:

- balance is private → cannot access directly
- Use methods → controlled access

```
}
```

Explanation:

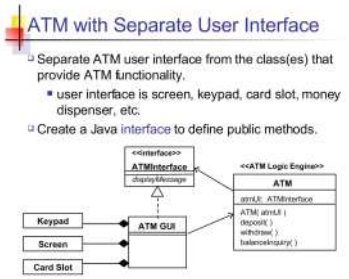
- balance is private → cannot access directly
- Use methods → controlled access

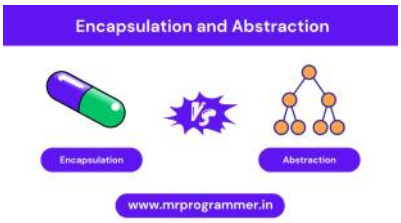
6. Abstraction

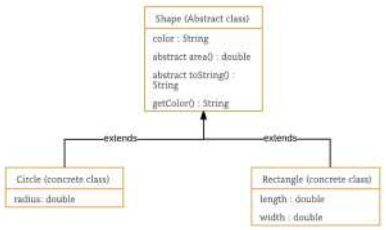
Definition

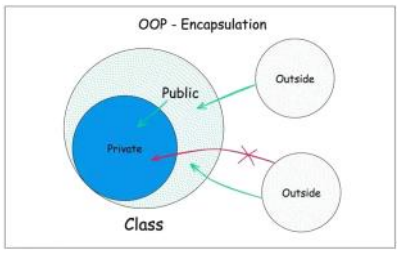
Hiding implementation details and showing only essential features.

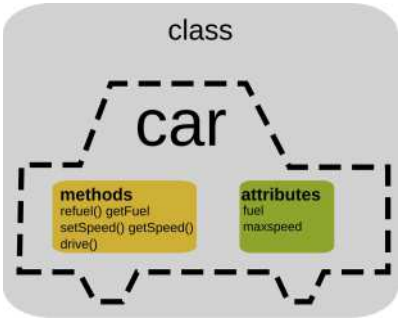












Example

```
abstract class Shape {
    abstract void draw(),
}
```

Example

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

Explanation:

- User doesn't know how drawing works internally
- Only calls draw()

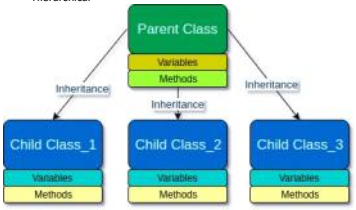
7. Inheritance

Definition

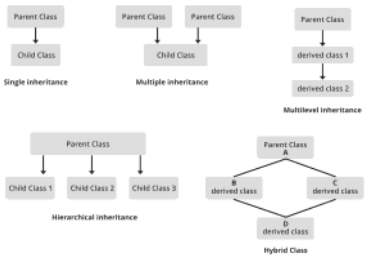
One class acquires properties of another.

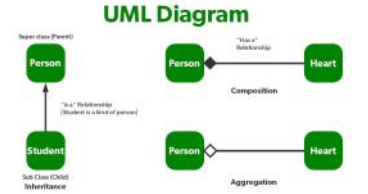
Types

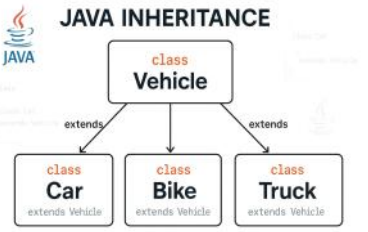
- Single
- Multilevel
- Hierarchical

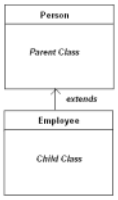




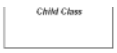








Example



Example

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
Dog d = new Dog();
d.eat();
d.bark();
```

8. Polymorphism

Definition

One name, multiple forms.

Types

1. Compile-time (Method Overloading)
2. Runtime (Method Overriding)

Method Overloading

```
class MathUtils {
    int add(int a, int b) {
        return a + b;
    }
int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}
```

9. Constructors

Definition

Special method used to initialize objects.

Types

- Default Constructor
- Parameterized Constructor

```
class Student {
    String name;
Student(String n) {
        name = n;
    }
}
Student s1 = new Student("Shivansh");
```

10. this Keyword

Used to refer to current object.

```
class Student {
    String name;
Student(String name) {
        this.name = name;
    }
}
```

11. static Keyword

Belongs to class, not object.

```
class Counter {
    static int count = 0;
Counter() {
        count++;
    }
}
```

12. Access Modifiers

Modifier	Scope
public	everywhere
private	within class
protected	within package + subclass
default	within package

13. Interfaces

Definition

Blueprint with only abstract methods.

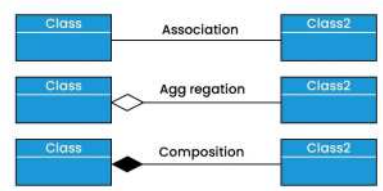
```
interface Animal {
    void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
```

14. Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Only abstract (mostly)
Multiple Inheritance	No	Yes
Constructor	Yes	No

15. Association, Aggregation, Composition

1. Association (General Relationship)



Definition

Association is a **general relationship** between two classes where they are connected but **independent**.

Key Points

- No ownership
- Objects can exist separately
- Can be **one-to-one**, **one-to-many**, or **many-to-many**

Example

Teacher ↔ Student

- Teacher teaches students
- Student can exist without teacher
- Teacher can exist without specific student

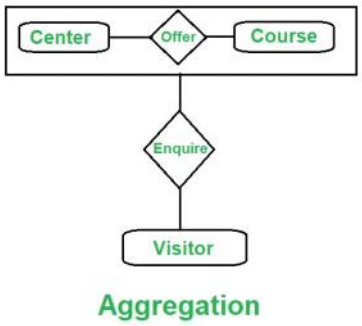
Code Example

```
class Teacher {
    String name;
}

class Student {
    String name;
}
```

☞ No strong dependency, just relationship.

2. Aggregation (Weak Relationship)



Aggregation

Definition
Aggregation is a **"has-a" relationship**, but **weak ownership**.

Key Points
- Child can exist independently
- Relationship is loose
- Represented as **hollow diamond (UML)**

Your Example Explained

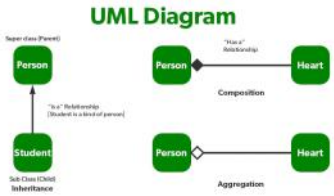
```
class Engine {}

class Car {
    Engine engine;
}
```

Explanation
- Car **has an Engine**
- But Engine can exist without Car
- Engine is **not created inside Car**
- It can be shared or reused

👉 Example:
- One engine can be used in different cars (conceptually)
- Removing car does NOT destroy engine

3. Composition (Strong Relationship)



Definition
Composition is a **strong "has-a" relationship** with **ownership**.

Key Points
- Child **cannot exist independently**
- Lifecycle is dependent
- Represented as **filled diamond (UML)**

Your Example Explained

```
class Heart {}

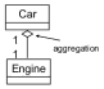
class Human {
    private Heart heart = new Heart();
}
```

Explanation
- Human **has a Heart**
- Heart is **created inside Human**
- If Human is destroyed → Heart is also destroyed

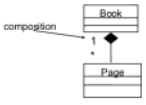
👉 Example:
- You cannot have a heart floating independently (logically in system design)
- Heart belongs strictly to that Human object

Association types

- **aggregation**: "is part of" / "has a"
  - symbolized by a clear white diamond
  - refers to the formation of a particular class as a result of one class being aggregated or built as a collection.



- **composition**: "is entirely made of"
  - stronger version of aggregation
  - the parts live and die with the whole
  - symbolized by a black diamond



15

4. Quick Comparison Table

Feature	Association	Aggregation	Composition
Relationship	General	Weak "has-a"	Strong "has-a"
Ownership	No	Partial	Full
Dependency	No	Low	High
Lifecycle	Independent	Independent	Dependent
Example	Teacher-Student	Car-Engine	Human-Heart

5. One-Line Memory Trick
- **Association** → Just connected
- **Aggregation** → Has-a (weak)
- **Composition** → Has-a (strong, owned)

- **Association** → Just connected
- **Aggregation** → Has-a (weak)
- **Composition** → Has-a (strong, owned)

16. Object Lifecycle

1. Object created → constructor called
2. Object used
3. Object destroyed → garbage collection

17. Garbage Collection

- Automatic memory management
- Removes unused objects

18. Real-world Example (Mini Project)

```
class User {
    String name;
User(String name) {
        this.name = name;
    }
void login() {
        System.out.println(name + " logged in");
    }
}
class Admin extends User {
    Admin(String name) {
        super(name);
    }
void deleteUser() {
        System.out.println("User deleted");
    }
}
```

19. Benefits of OOP

- Code reusability
- Scalability
- Maintainability
- Modularity
- Security

20. When to Use OOP

- Large-scale systems
- Real-world modeling
- Backend systems
- Game development

OOP — Top 20 Most Asked Interview Questions

1. What is OOP?

OOP (Object-Oriented Programming) is a paradigm based on objects that contain data (attributes) and methods (functions).

2. What are the four pillars of OOP?

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

3. What is a class?

A blueprint/template used to create objects.

4. What is an object?

An instance of a class with real values.

5. Difference between class and object?

- Class → blueprint
- Object → real instance created from class

6. What is encapsulation?

Wrapping data and methods together and restricting direct access using access modifiers.

7. What is abstraction?

Hiding internal implementation and showing only essential functionality.

8. Difference between abstraction and encapsulation?

- Abstraction → hides implementation
- Encapsulation → hides data

9. What is inheritance?

Mechanism where one class acquires properties of another class.

10. Types of inheritance?

- Single
- Multilevel
- Hierarchical
- Multiple (via interfaces in Java)

11. What is polymorphism?

Ability of a function/method to take multiple forms.

12. Types of polymorphism?

- Compile-time → Method overloading
- Runtime → Method overriding

13. What is method overloading?

Same method name with different parameters (compile-time polymorphism).

14. What is method overriding?

Redefining a parent class method in a child class (runtime polymorphism).

13. What is method overloading?
Same method name with different parameters (compile-time polymorphism).

14. What is method overriding?
Redefining a parent class method in a child class (runtime polymorphism).

15. What is a constructor?
Special method used to initialize objects when they are created.

16. What is the this keyword?
Refers to the current object of the class.

17. What is the static keyword?
Belongs to the class rather than instance (shared across all objects).

18. What is an interface?
A blueprint containing abstract methods that a class must implement.

19. Difference between abstract class and interface?
- Abstract class → can have both abstract & concrete methods
- Interface → mostly only abstract methods (Java modern allows default/static)

20. What is the difference between aggregation and composition?
- Aggregation → weak relationship (can exist independently)
- Composition → strong relationship (dependent lifecycle)